

# Exercise Solutions Set 1: Master Analysis & Teacher Commentary

## Exhaustive Step-by-Step Tracing, Code Analysis & Hardware Memory Rules

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Instructor: Prof. Brandon W. Gordon · Course: MIAE 215 / MECH 215 · Language: Standard C++

### Overview

This document presents a comprehensive, line-by-line analysis of **Official Exercise Solutions Set 1** provided by Prof. Gordon. It integrates the exact program outputs and technical commentaries from the professor's logs ( `output_with_comments.txt` ), highlighting crucial traps that frequently appear on midterm and final examinations.

#### Exercise Solutions Set 1

- └─ Module 1: Variable Types 1 (Type declaration, bounds, corrected program)
- └─ Module 2: Variable Types 2 (ASCII character math, casing, explicit casting)
- └─ Module 3: Expressions & Operators (Modulo digit peeling, IEEE 754 precision, trig limits)

## Module 1: Variable Types 1 Solutions

### 1. Corrected Code Analysis ( `variable_types1_solutions_corrected.cpp` )

The original exercise contains common beginner mistakes: variable name collisions, missing headers, uninitialized variables, and overflow errors.

#### Key Code Tracing:

```
#include <cstdio>
#include <iostream>

using namespace std;

int main() {
    int i = 0;
    double x = 10.0;
    double y = 20.0;
    double z = x + y;

    cout << "z = " << z << "\n";
    // Output: z = 30
    return 0;
}
```

## 2. Teacher Commentary & Hardware Trap

[!IMPORTANT] **Uninitialized Local Variables:** In C++, local variables allocated on the stack are not automatically cleared. If a variable is read before it is assigned, the CPU reads whatever residual bit pattern previously occupied that memory slot ("garbage").

- Always initialize upon declaration: `int count = 0; , double sum = 0.0; .`

## Module 2: Variable Types 2 Solutions

### Question 1: ASCII Character Manipulation & Case Conversion

C++ stores `char` data as 1-byte integers corresponding to standard ASCII codes (0 to 127).

#### 1. Teacher Output Log & Line Tracing:

```
c1 = A (ASCII 65)
c2 = a (ASCII 97)
Difference = 32
c3 = b (ASCII 98) -> Upper: B (ASCII 66)
```

#### 2. The Core ASCII Conversion Formulas:

| Goal                   | C++ Code Expression           | Numerical Operation               |
|------------------------|-------------------------------|-----------------------------------|
| Lowercase to Uppercase | <code>upper = ch - 32;</code> | $97 - 32 = 65$ ( 'a' ) to ( 'A' ) |
| Uppercase to Lowercase | <code>lower = ch + 32;</code> | $65 + 32 = 97$ ( 'A' ) to ( 'a' ) |
| Char Digit to Integer  | <code>val = ch - '0';</code>  | $53 - 48 = 5$ ( '5' ) to ( 5 )    |
| Integer to Char Digit  | <code>ch = val + '0';</code>  | $5 + 48 = 53$ ( 5 ) to ( '5' )    |

#### 3. Narrowing Cast Truncation:

When a multi-byte integer is cast into a 1-byte `char`, only the lowest 8 bits are preserved:

```
int x = 321; // Binary: 00000001 01000001 (lowest byte is 01000001 = 65 in dec)
char ch = (char)x;
cout << ch; // Prints: 'A' (ASCII 65)!
```

## Module 3: Expressions & Operators Solutions

### 1. Extended Multi-Assignment & Initialization

```
int x, y, z, q;
x = y = z = q = 10;
// Evaluates right-to-left. All variables receive 10.
```

### 2. Floating-Point Limits: Overflow & Underflow

```
double x = 1.0e308;
double y = 1.0e308;
double z = 1.0e-306;

x = x * 10.0; // Overflows double capacity -> inf
y = y * 1.0e10; // Overflows -> inf
z = z / 1.0e10; // Underflows below minimum positive normalized limit -> 0.0
```

### 3. Alternating Sequences with Integer Division

```
int j = -1;
for (int k = 0; k < 10; k++) {
    j = -j;
    cout << "j = " << j << "\n";
}
// Outputs alternating: 1, -1, 1, -1, 1, -1...
```

### 4. Right-to-Left Decimal Digit Peeling ( $b = 237$ )

```
int b = 237;
int r;

r = b % 10; // r = 7 (Units digit)
b = b / 10; // b becomes 23

r = b % 10; // r = 3 (Tens digit)
b = b / 10; // b becomes 2

r = b % 10; // r = 2 (Hundreds digit)
```

[!NOTE] **Prof. Gordon's Direct Note:** "Note: 7, 3, 2 is the opposite digit order of  $b$  (237) — using remainder with repeated division by 10 is the standard algorithm to extract and reverse digits of an integer."

### 5. Mixed Variable Type Arithmetic

```
double d1, d2, d3;
int i1 = 3;

d1 = 7.1;
d2 = 31.0 / 3.0; // 10.333333...
d3 = 21 / 3;     // Integer division 7, widened to 7.0
```

### 6. Trigonometric Library Functions in Base 2 ( `<cmath>` )

```
const double PI = 3.141592653589793;
double x = PI / 2.0; // 90 degrees = 1.5707963 rad

cout << "sin(x) = " << sin(x) << "\n"; // 1.0
cout << "cos(x) = " << cos(x) << "\n"; // 6.12323e-17
cout << "tan(x) = " << tan(x) << "\n"; // 1.63312e+16
```

#### Detailed Precision Breakdown:

1.  $\cos(\pi/2) = 6.12323 \times 10^{-17}$ :
  - This is equivalent to 0.0 to within 16 decimal places (0.0000000000000000612...).
  - *Why not zero?* Because  $\pi$  cannot be stored exactly in binary floating-point representation, and trigonometric functions use Taylor/Chebyshev polynomial approximations.
2.  $\tan(\pi/2) = 1.63312 \times 10^{16}$ :
  - Evaluates to an enormous number ( $\sim 16$  quadrillion) rather than `+inf` because  $\cos(\pi/2)$  is an infinitesimally small positive number rather than exact 0.0:

$$\tan\left(\frac{\pi}{2}\right) = \frac{\sin(\pi/2)}{\cos(\pi/2)} = \frac{1.0}{6.12323 \times 10^{-17}} \approx 1.63312 \times 10^{16}$$

## Master Exam Cheatsheet for Problem Set 1

1. **Integer Division:** Always verify both operands. `3 / 4 == 0`, but `3.0 / 4 == 0.75`.
2. **Modulo Limitations:** Never apply `%` to `float` or `double`. It is exclusively for integer types (`int`, `char`, `short`, `long`). For floating-point remainder, `<cmath>` provides `fmod(x, y)`.
3. **Float Equality Rule:** Never write `if (cos(x) == 0.0)`. Always write `if (abs(cos(x)) < 1.0e-7)`.
4. **Digit Extraction Formula:**
  - Current Digit =  $n \text{ (mod } 10)$
  - Shift Leftward =  $n/10$
5. **Chained Postfix Evaluation:** In `cout << q++ << " " << q;`, behavior is compiler-dependent. Always increment on a separate line!